Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

IIC2115 - Programación como herramienta para la ingeniería

Manejo y análisis de datos tabulares

Profesor: Hans Löbel

¿Qué es el análisis de datos en Python?

- A través de este análisis, buscamos responder preguntas relevantes, o descubrir aspectos desconocidos, en base a la evidencia dada por los datos
- Inicialmente exploraremos los datos tabulares, para luego pasar a otros tipos de datos en el resto del curso.
- Desde un punto de vista práctico, buscaremos realizar las siguientes tareas:
    - Exploración de distintas dimensiones
    - Limpieza, corrección y transformación
    - Análisis estadístico

Para todo esto, utilizaremos Pandas

- Biblioteca de Python que permite manipular, analizar y visualizar datos.
- Puede ser visto como una herramienta para trabajar datos almacenados en una estructura de tabla o de serie de tiempo.
- Se basa en, y generaliza a, la biblioteca Numpy.
- 2 Estructuras principales
  - Series
  - DataFrame

pandas

En un DataFrame, cada columna es un Series

	Comuna	Manzana	Predial	Línea de construcción	Material estructural	Calidad construcción	Año construcción
0	9201	1	1	1	E	4	1940
1	9201	1	1	2	E	4	1960
2	9201	1	2	1	E	4	1930
3	9201	1	3	1	E	4	1960
4	9201	1	4	1	E	3	1925

El primero paso siempre es explorar los datos

- Abrirlos
- Describirlos (verlos)
- Calcular estadísticos básicos que faciliten los siguientes pasos

El primero paso siempre es explorar los datos

```python
import pandas as pd
import numpy as np

df = pd.read_csv("data.csv")
```

```python
display(df.head(10))
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Property_Area	Loan_Status
0	LP001002	Male	No	0	Graduate	No	5849	0.0	NaN	360.0	1.0	Urban	Y
1	LP001003	Male	Yes	1	Graduate	No	4583	1508.0	128.0	360.0	1.0	Rural	N
2	LP001005	Male	Yes	0	Graduate	Yes	3000	0.0	66.0	360.0	1.0	Urban	Y
3	LP001006	Male	Yes	0	Not Graduate	No	2583	2358.0	120.0	360.0	1.0	Urban	Y
4	LP001008	Male	No	0	Graduate	No	6000	0.0	141.0	360.0	1.0	Urban	Y
5	LP001011	Male	Yes	2	Graduate	Yes	5417	4196.0	267.0	360.0	1.0	Urban	Y
6	LP001013	Male	Yes	0	Not Graduate	No	2333	1516.0	95.0	360.0	1.0	Urban	Y
7	LP001014	Male	Yes	3+	Graduate	No	3036	2504.0	158.0	360.0	0.0	Semiurban	N
8	LP001018	Male	Yes	2	Graduate	No	4006	1526.0	168.0	360.0	1.0	Urban	Y
9	LP001020	Male	Yes	1	Graduate	No	12841	10968.0	349.0	360.0	1.0	Semiurban	N

```python
df.describe()
```

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History
count	614.000000	614.000000	592.000000	600.00000	564.000000
mean	5403.459283	1621.245798	146.412162	342.00000	0.842199
std	6109.041673	2926.248369	85.587325	65.12041	0.364878
min	150.000000	0.000000	9.000000	12.00000	0.000000
25%	2877.500000	0.000000	100.000000	360.00000	1.000000
50%	3812.500000	1188.500000	128.000000	360.00000	1.000000
75%	5795.000000	2297.250000	168.000000	360.00000	1.000000
max	81000.000000	41667.000000	700.000000	480.00000	1.000000

```python
property_area = df['Property_Area']
display(property_area)
```

```
0          Urban
1          Rural
2          Urban
3          Urban
4          Urban
         ...
609        Rural
610        Rural
611        Urban
612        Urban
613    Semiurban
Name: Property_Area, Length: 614, dtype: object
```

```python
property_area.value_counts()
```

```
Semiurban    233
Urban        202
Rural        179
Name: Property_Area, dtype: int64
```

A continuación, podemos comenzar a procesar los datos

- Identificar la presencia de valores nulos y *outliers*
- Corregir y/o imputar lo que corresponda

```python
def conteo_nulo(x):
    return sum(x.isnull())

df.apply(conteo_nulo, axis = 0)
```

```
Loan_ID               0
Gender               13
Married               3
Dependents           15
Education             0
Self_Employed        32
ApplicantIncome       0
CoapplicantIncome     0
LoanAmount           22
Loan_Amount_Term     14
Credit_History       50
Property_Area         0
Loan_Status           0
dtype: int64
```

```python
def conteo_nulo(x):
    return sum(x.isnull())

df.apply(conteo_nulo, axis = 0)
```

```
Loan_ID               0
Gender               13
Married               3
Dependents           15
Education             0
Self_Employed        32
ApplicantIncome       0
CoapplicantIncome     0
LoanAmount           22
Loan_Amount_Term     14
Credit_History       50
Property_Area         0
Loan_Status           0
dtype: int64
```

```python
df['LoanAmount'].fillna(df['LoanAmount'].mean(), inplace=True)
```

```python
df.apply(conteo_nulo, axis = 0)
```

```python
def conteo_nulo(x):
    return sum(x.isnull())

df.apply(conteo_nulo, axis = 0)
```

```
Loan_ID               0
Gender               13
Married               3
Dependents           15
Education             0
Self_Employed        32
ApplicantIncome       0
CoapplicantIncome     0
LoanAmount           22
Loan_Amount_Term     14
Credit_History       50
Property_Area         0
Loan_Status           0
dtype: int64
```

```python
df['LoanAmount'].fillna(df['LoanAmount'].mean(), inplace=True)
```

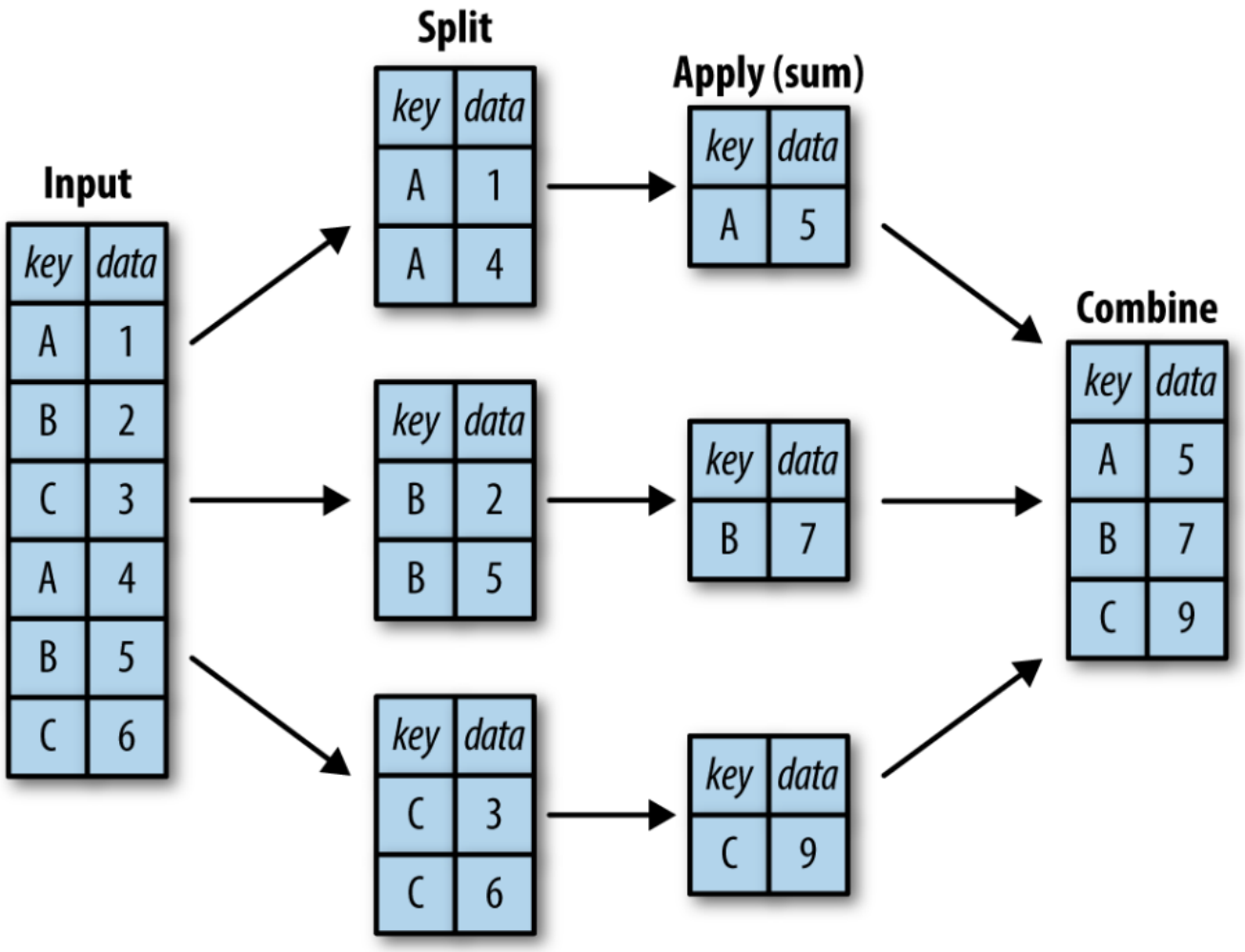
```python
df.apply(conteo_nulo, axis = 0)
```

```
Loan_ID               0
Gender               13
Married               3
Dependents           15
Education             0
Self_Employed        32
ApplicantIncome       0
CoapplicantIncome     0
LoanAmount            0
Loan_Amount_Term     14
Credit_History       50
Property_Area         0
Loan_Status           0
dtype: int64
```

Agregación es típicamente el primer análisis estadístico

- Analizar tendencias o buscar patrones se hace difícil si el análisis es individual
- Para evitar esto, datos generalmente se analizan de manera agregada
- Además de esto, la agregación suele ser a nivel grupal y no global
- Pandas permite enfrentar estos problemas con una serie de mecanismos que facilitan la exploración

Función groupby permite combinar todo el procesamiento agregado



Pandas provee múltiples funciones de agregación

Aggregation	Description
`count()`	Total number of items
`first()`, `last()`	First and last item
`mean()`, `median()`	Mean and median
`min()`, `max()`	Minimum and maximum
`std()`, `var()`	Standard deviation and variance
`mad()`	Mean absolute deviation
`prod()`	Product of all items
`sum()`	Sum of all items

Veamos algunos ejemplos de Pandas con datos reales

```python
import numpy as np
import pandas as pd
import seaborn as sns
titanic = sns.load_dataset('titanic')
titanic.head()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True

¿Es cierta la expresión de esa época “mujeres y niños primero”?

¿Es cierta la expresión de esa época “mujeres y niños primero”?

```
1 titanic.groupby('sex')[['survived']].mean()
```

sex	survived
female	0.742038
male	0.188908

¿Es cierta la expresión de esa época “mujeres y niños primero”?

```python
titanic.groupby('sex')[['survived']].mean()
```

sex	survived
female	0.742038
male	0.188908

```python
titanic.groupby(['sex', 'class'])['survived'].aggregate('mean')
```

```
sex     class
female  First     0.968085
        Second    0.921053
        Third     0.500000
male    First     0.368852
        Second    0.157407
        Third     0.135447
Name: survived, dtype: float64
```

Tablas dinámicas son otra forma de agrupar

- Nos permiten agregar valores utilizando múltiples dimensiones y funciones.
- Además, Pandas permite crear rangos para las variables a analizar, permitiendo un análisis más fino.

Tablas dinámicas nos permiten simplificar y extender la agrupación y agregación

```python
titanic.pivot_table('survived', index='sex', columns='class')
```

class	First	Second	Third
sex			
female	0.968085	0.921053	0.500000
male	0.368852	0.157407	0.135447

```python
age = pd.cut(titanic['age'], [0, 18, 80])
titanic.pivot_table('survived', ['sex', age], 'class')
```

	class	First	Second	Third
sex	age			
female	**(0, 18]**	0.909091	1.000000	0.511628
	(18, 80]	0.972973	0.900000	0.423729
male	**(0, 18]**	0.800000	0.600000	0.215686
	(18, 80]	0.375000	0.071429	0.133663

Tablas dinámicas nos permiten simplificar y extender la agrupación y agregación

```python
fare = pd.qcut(titanic['fare'], 2)
titanic.pivot_table('survived', ['sex', age], [fare, 'class'])
```

	fare	(-0.001, 14.454]			(14.454, 512.329]		
	class	First	Second	Third	First	Second	Third
sex	age						
female	**(0, 18]**	NaN	1.000000	0.714286	0.909091	1.000000	0.318182
	(18, 80]	NaN	0.880000	0.444444	0.972973	0.914286	0.391304
male	**(0, 18]**	NaN	0.000000	0.260870	0.800000	0.818182	0.178571
	(18, 80]	0.0	0.098039	0.125000	0.391304	0.030303	0.192308

```python
titanic.pivot_table(index='sex', columns='class',
                    aggfunc={'survived':sum, 'fare':'mean'})
```

	fare			survived		
class	First	Second	Third	First	Second	Third
sex						
female	106.125798	21.970121	16.118810	91	70	72
male	67.226127	19.741782	12.661633	45	17	47

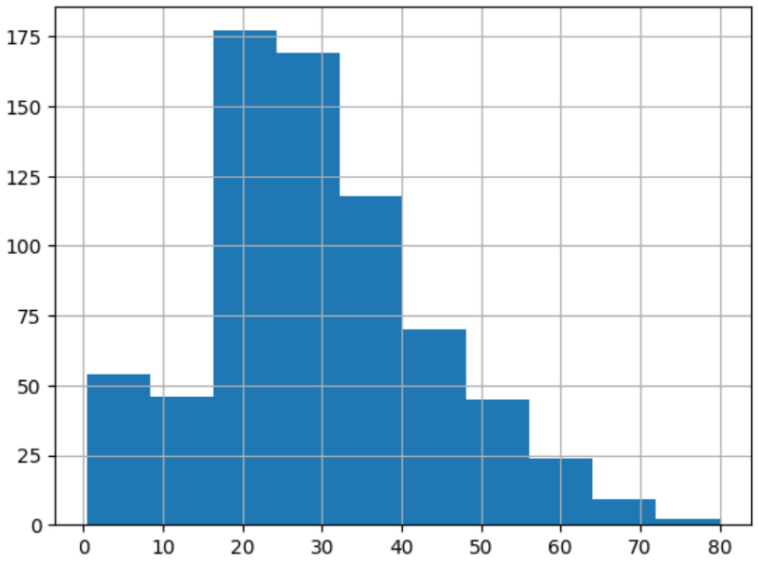
Podemos complementar el análisis a través de la visualización

- Existen varias maneras en Python de presentar resultados gráficamente. Todas comparten la facilidad de uso y gran calidad de la presentación.

- Con el fin de facilitar su uso, Pandas incorpora varias visualizaciones adecuadas a Series y DataFrame.

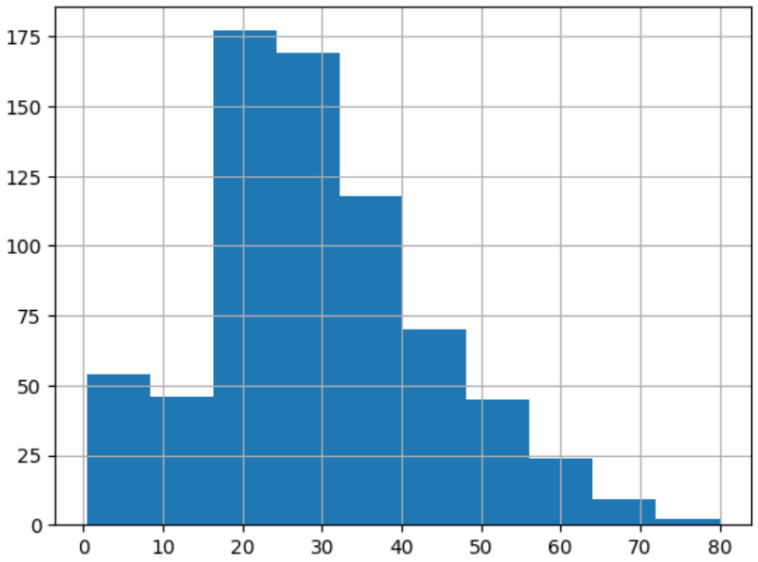
Un ejemplo de la utilidad de la visualización puede verse al buscar *outliers*

```
titanic['age'].hist();
```

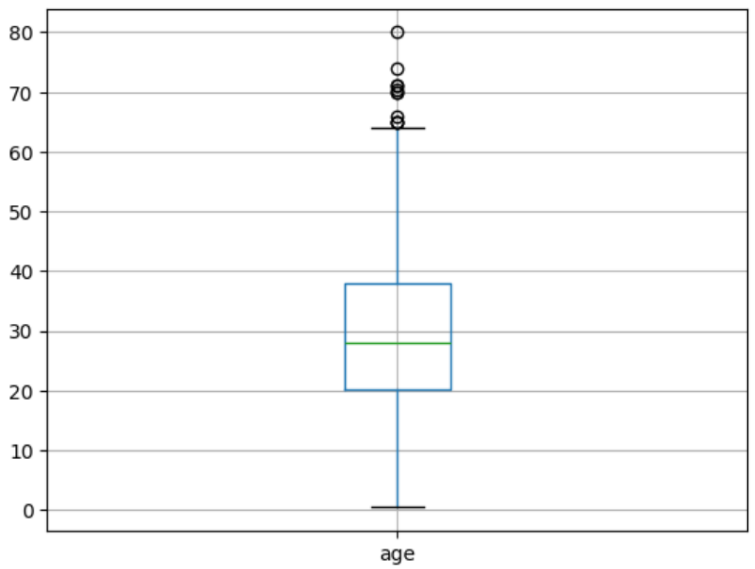


Un ejemplo de la utilidad de la visualización puede verse al buscar *outliers*

```
1 titanic['age'].hist();
```

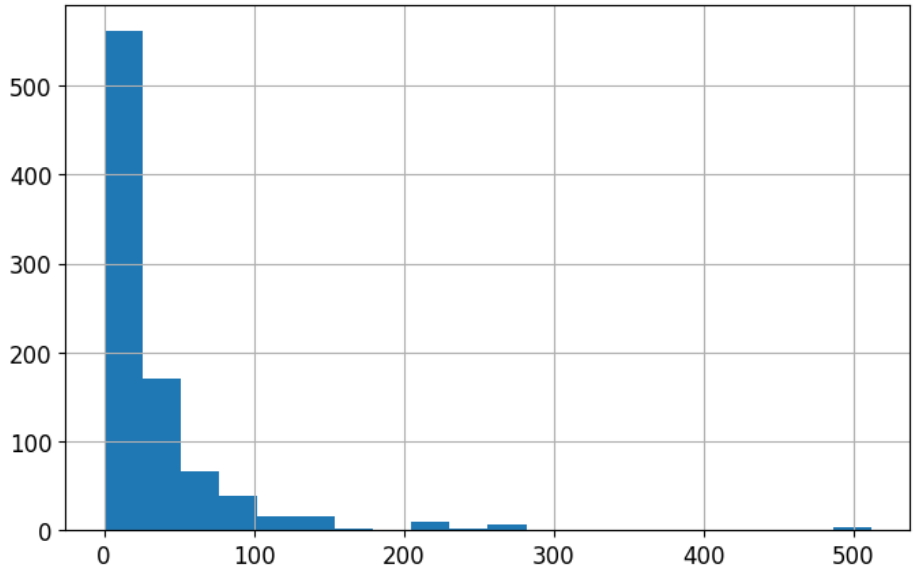



```
1 titanic.boxplot(column='age');
```



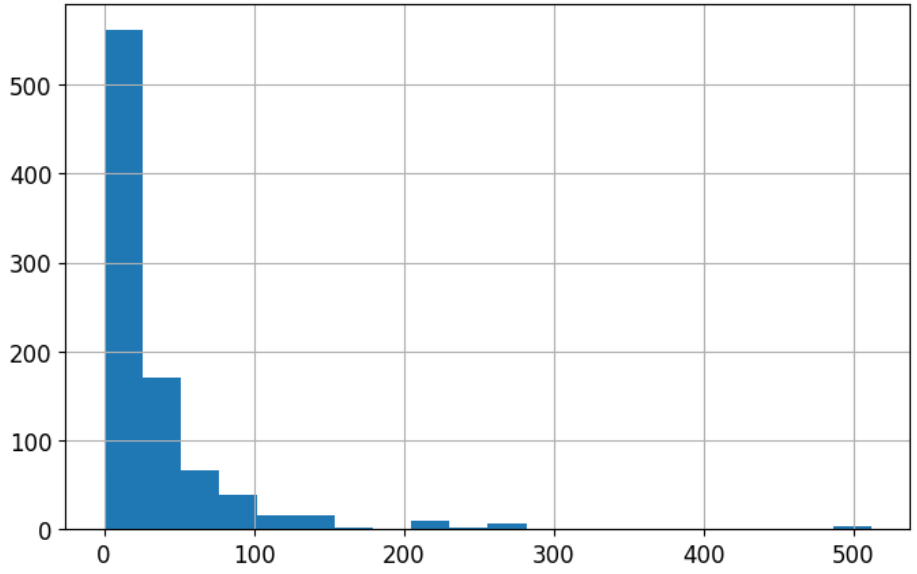
Un ejemplo de la utilidad de la visualización puede verse al buscar *outliers*

```
1 titanic['fare'].hist(bins=20);
```

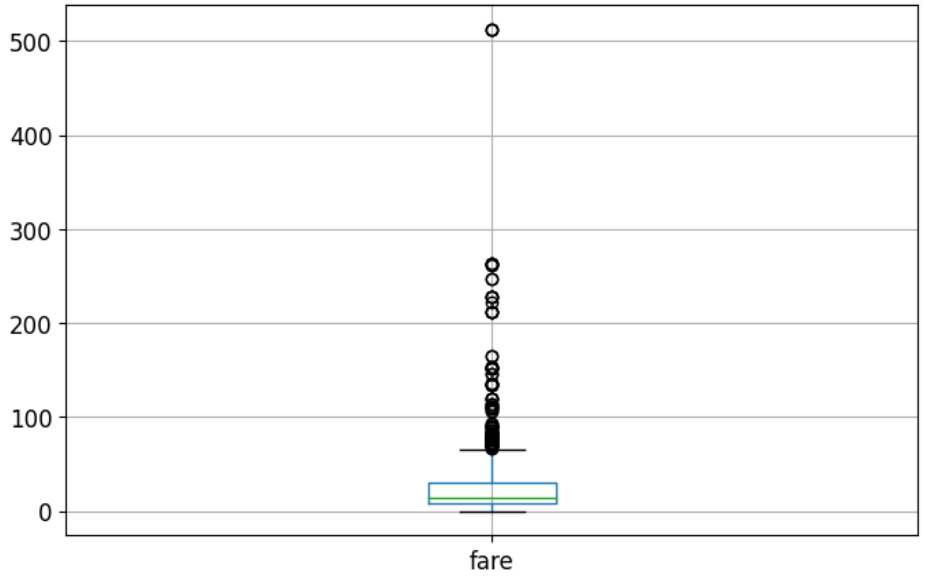


Un ejemplo de la utilidad de la visualización puede verse al buscar *outliers*

```
titanic['fare'].hist(bins=20);
```




```
titanic.boxplot(column='fare');
```



Cómo sigue la sesión

- Para este capítulo hay 3 ejercicios formativos disponibles.
- El primer ejercicio tiene formato tutorial, con el objetivo de cubrir la materia.
- El segundo es más avanzado les permitirá practicar múltiples comandos y técnicas.
- El tercero es más realista y servirá para preparar el laboratorio
- No olviden responder el ticket de salida, siempre considerando que lo indicado en él debe verse reflejado en el repositorio privado.
- A las 17:30 se cerrará el ticket de manera definitiva.

Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

IIC2115 - Programación como herramienta para la ingeniería

Manejo y análisis de datos tabulares

Profesor: Hans Löbel